

Lecture 1: Git & VS Code (& NumPy)

TECH2: Introduction to Programming, Data, and Information Technology

Richard Foltyn
NHH Norwegian School of Economics

September 30, 2026

See GitHub repository for notebooks and data:

<https://github.com/richardfoltyn/TECH2-H26>

Contents

1	Benchmarking against NumPy's <code>argmax()</code>	1
1.1	Benchmarking for 10 values	2
1.2	Benchmarking for 1,000 values	2

1 Benchmarking against NumPy's `argmax()`

We want to compare our implementation of `argmax()` against NumPy's own implementation, `np.argmax()`.

1. Import the `lecture1` module and NumPy.
2. Use the function `get_test_values()` defined below to get a sample of 10 randomly drawn numbers.
3. Use the `%timeit` cell magic to measure the execution time of your `argmax()` implementation against NumPy's. This can be achieved as follows:

```
%timeit np.argmax(values)
```

Which one is faster?

4. Repeat the exercise for a sample of 1,000 random numbers.

```
[1]: def get_test_values(n=10):  
    """  
    Return an array of n randomly generated values.  
  
    Parameters  
    -----  
    n : int  
        The number of random values to generate (default: 10).  
  
    Returns  
    -----  
    np.ndarray  
        An array of n random values uniformly distributed in the range [0, 1].  
    """  
    import numpy as np
```

```

# Create an instance of a random number generator
rng = np.random.default_rng(seed=1234)

# Generate n random values uniformly distributed in the range [0, 1]
values = rng.random(n)

return values

```

The following lines enable automatic reloading of modules outside of this notebook if these change. Lines starting with % are so-called “cell magics” which only work in Jupyter notebooks, but are not valid Python syntax.

```

[2]: %load_ext autoreload
      %autoreload 2

```

```

[3]: # Import module with our implementation of argmax()
      import lecture1

      # Import NumPy
      import numpy as np

```

1.1 Benchmarking for 10 values

```

[4]: # Draw 10 random numbers on the interval [0, 1]
      values = get_test_values(10)

```

```

[5]: # Benchmark NumPy's argmax()
      %timeit np.argmax(values)

```

690 ns ± 5.9 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)

```

[6]: # Benchmark our argmax()
      %timeit lecture1.argmax(values)

```

592 ns ± 4.38 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)

1.2 Benchmarking for 1,000 values

```

[7]: # Draw 1000 random numbers on the interval [0, 1]
      values = get_test_values(1000)

```

```

[8]: # Benchmark NumPy's argmax()
      %timeit np.argmax(values)

```

775 ns ± 4.07 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)

```

[9]: # Benchmark our argmax()
      %timeit lecture1.argmax(values)

```

55.5 µs ± 552 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)